hashlock.

Security Audit

helder (DeFi)

April 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	20
Conclusion	21
Our Methodology	22
Disclaimers	24
About Hashlock	25

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The helder team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

helder is a cutting-edge DeFi staking protocol on Binance Smart Chain offering hYBRID compounding and per-second auto-staking to maximize passive income opportunities. Its tokenomics feature a limited supply of 500 million $helder tokens with a 0.50% auto-burn mechanism that can burn up to 90% of transaction fees, ensuring sustainable deflationary growth. The protocol's smart contracts are fully verified and audited, and the project is backed by a fully doxxed team to guarantee transparency and security. With 100% of liquidity pool tokens burned and anti-rug pull safeguards in place, helder provides a secure environment for investors seeking reliable yield strategies. Stakers can earn fixed APYs of up to +100% while enjoying automatic hYBRID compounding that reinvests rewards directly into their wallets every second. The protocol also includes a referral system that rewards users for bringing new participants, further enhancing user-driven growth and community engagement. helder's presale is scheduled for 25th April 2025 on the qerra hYBRID Launchpad Ecosystem, marking the first major opportunity to join its exclusive staking experience.

Project Name: helder

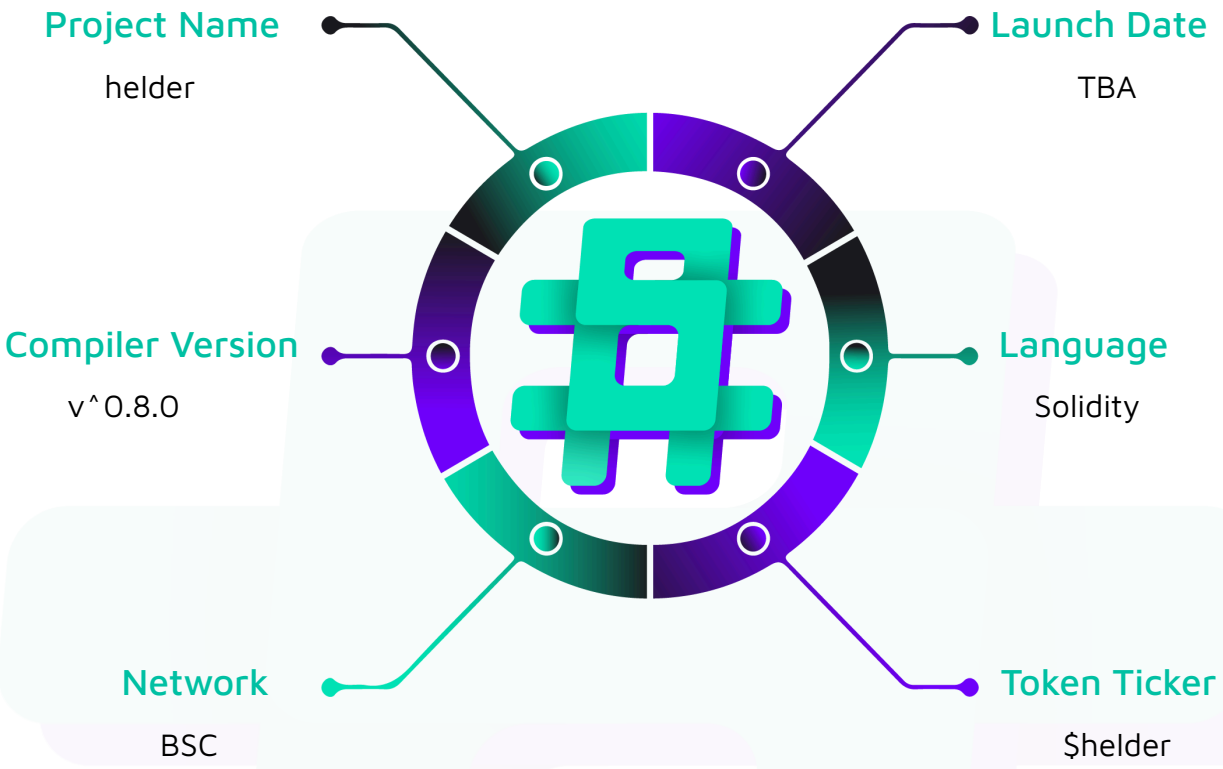
Project Type: Defi, Staking

Compiler Version: ^0.8.0

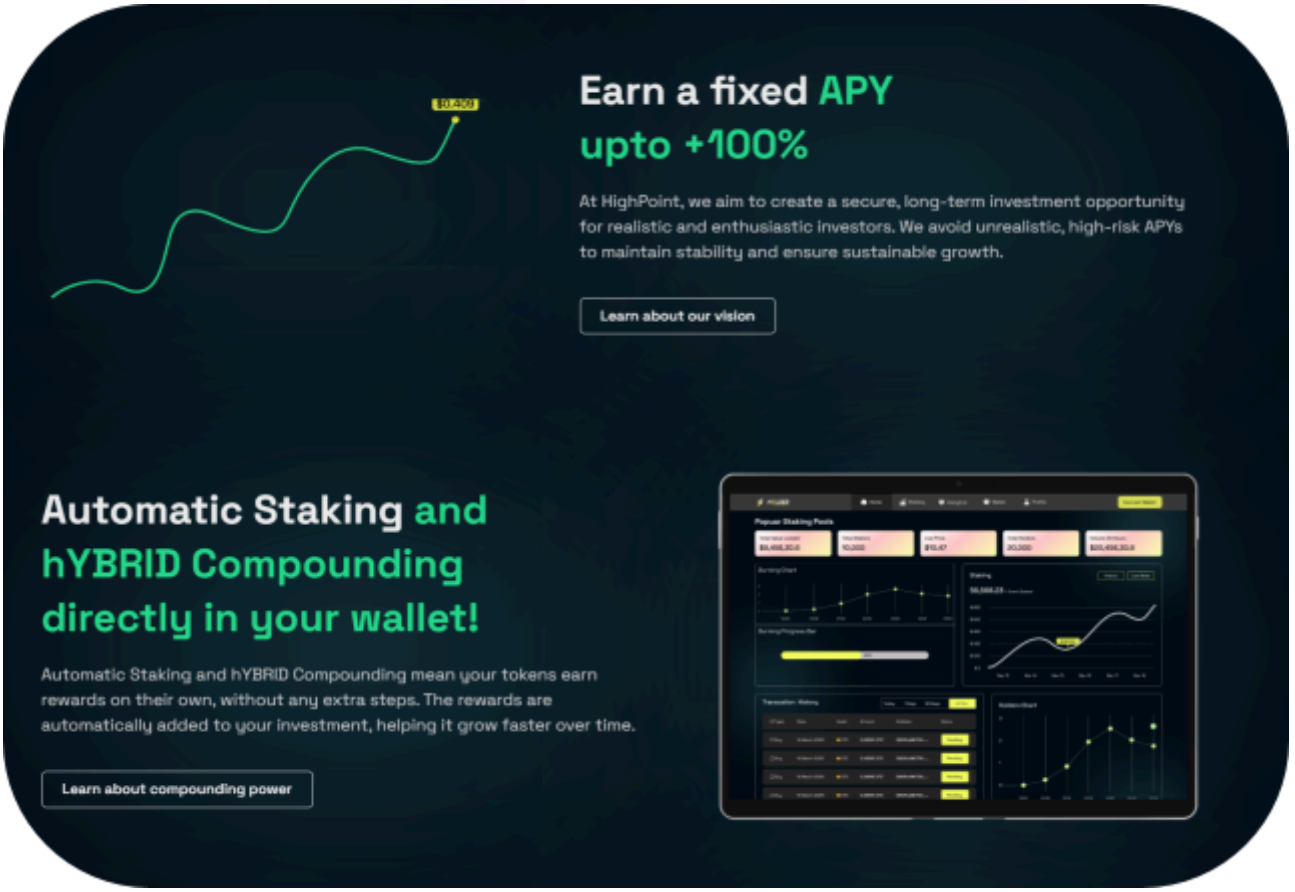
Website: https://www.helder.world/

Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the helder project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	helder Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	April, 2025
Contract 1	helderTokenContract.sol
Contract 1 MD5 Hash	fa5e272c5fab3a295f91567ec4ac3074
Deployed Mainnet Address	
Deployed Network/Chain	

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

1 Gas Optimization

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Helder Token Contract.sol - Allows users to: - Transfer tokens with 0.5% fee - Approve and transfer tokens on behalf of others(standard ERC-20) - View total burned tokens - Allows Admins to: - Distribute contract's token balance to core members with 0.5% burn - Allows Owner to: - Mint initial helder token supply to recipient - Add core members - Remove core members	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the helder project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the helder project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] helder-ERC20#constructor - Lack of Owner Address Validation Leads to Losing Access and Deployment Issues

Description

The contract fails to validate the `_owner` address during initialization. If an invalid address (e.g., `address(0)` or an incorrect address) is provided during deployment, the contract lacks a mechanism to update or recover the owner role. This could permanently lock administrative functions such as `addCoreT` and `removeCoreT`, rendering them unusable.

Impact

If an incorrect address is set as the owner during deployment, this could result in permanent loss of administrative control and critical owner-only functions will be inaccessible.

Recommendation

During construction, set the `_owner` to `msg.sender` so by this, there won't be an issue with address being null-address or wrong address. Additionally, implement functions similar to Openzeppelin's `Ownable2Step` to change the `_owner` in the future by implementing functions like `transferOwnership` and `acceptOwnership`

Status

Resolved

[M-02] helder#renounceOwnership - Admin Role revocation Can permanently lock contract Functionality

Description

The helder contract relies solely on `Ownable2Step` for admin role management but lacks protection against complete admin role removal, which could permanently lock all admin-controlled functions.

The critical issue is that `_owner` can: Revoke their own role(using `renounceOwnership` function), which transfers the ownership to `address(0)` and leaves the contract without any admin. This affects `distributeFee` and other Owner-related functions:

```
function distributeFee()
    external
    onlyOwner
{
    // Function becomes permanently inaccessible
    // if all admins are removed
}
```

Impact

If the admin role is revoked either accidentally or maliciously, the helder becomes permanently unusable. This would lock all fee management functionality, including the ability to withdraw accumulated fees. There is no recovery mechanism possible without contract redeployment, making any funds in the helder permanently trapped.

Recommendation

Override the `renounceOwnership()` and make it nullify.

Status

Resolved

Low

[L-01] helder-ERC20#constructor - No Check for Duplicate Addresses in CoreMembers array

Description

When adding administrators to the `coreMembers` array, the contract does not check for duplicate addresses. If the same admin address appears multiple times in the `_admins` array, it will be added to the `coreMembers` array multiple times as there is no check to validate against the mapping to check for duplicates.

Impact

This could lead to unfair token distribution when calling `distributeFee`, as the same admin would receive multiple shares if their address appears multiple times in the array. Additionally, it would inflate the array unnecessarily, increasing gas costs for operations that iterate through all core members.

Recommendation

Implement a check to prevent duplicate entries in the `coreMembers` array by checking against the mapping – similar to how it is done in `addCoreT` and `removeCoreT` functions.

Status

Resolved

[L-02] helder#distributeFee - Dust Amounts were not properly managed in Fee Distribution Logic

Description

The `distributeFee` function divides the contract's token balance by the number of core members. This integer division can leave dust amounts (remainder tokens) that are not distributed to anyone.

Impact

Over time, these dust amounts could accumulate in the contract, leading to discrepancy between expected and actual token distribution. While the impact is minor for individual transactions, it could become significant with repeated operations.

Recommendation

Consider sending dust amounts by doing the appropriate math by multiplying the values with higher basis points. Or modify the given code as per your logic:

```
// Example approach to handle dust

function distributeFee() external onlyAdmin {

   uint256 contractBalance = balanceOf(address(this));

   uint256 memberCount = coreMembers.length;

   uint256 valuePerMember = contractBalance / memberCount;

   uint256 totalDistributed = 0;


   for (uint i = 0; i < memberCount; i++) {

       uint256 memberShare = valuePerMember;

       uint256 burningAmount = (memberShare * 5) / 1000;

       uint256 actualAmount = memberShare - burningAmount;


       _transfer(address(this), coreMembers[i], actualAmount);

       _burn(address(this), burningAmount);

       totalBurnToken += burningAmount;

       emit burnAmount(burningAmount);


       totalDistributed += memberShare;

   }
```

```
    // Handle any remaining dust at the end
    uint256 remainingDust = contractBalance - totalDistributed;
    if (remainingDust > 0) {
        // Option 1: Burn the dust
        _burn(address(this), remainingDust);
        totalBurnToken += remainingDust;
        emit burnAmount(remainingDust);

        // Option 2: Or distribute to the first member
        // _transfer(address(this), coreMembers[0], remainingDust);
    }
}
```

Status

Resolved

Gas

[G-01] helder - Unnecessary Variable Initialization

Description

The `totalBurnToken` state variable is explicitly initialised to 0, which is redundant as uint variables are automatically initialized to 0 in solidity by default.

Impact

This causes an unnecessary gas expense during contract deployment.

Recommendation

Remove the explicit initialization of the `totalBurnToken` variable

Status

Resolved

QA

[Q-01] helder - Redundant Inheritance

Description

The ERC20 contract inherits from both IERC20 and IERC20Metadata, but IERC20Metadata already inherits from IERC20, making the direct inheritance of IERC20 redundant

Recommendation

Simplify the inheritance by only inheriting from IER20Metadata (which already includes IERC20)

Status

Resolved

[Q-02] helder - Non-specific Solidity Complier Version

Description

The contract uses a floating pragma statement (^0.8.0), which means it can be compiled with any 0.8.x version of solidity. Different compiler versions may have different bugs and security fixes. Using a floating pragma could potentially introduce inconsistent behaviour across deployments/testing.

Recommendation

Lock the pragma to a specific Solidity version to ensure consistency compilation and behavior.

Status

Resolved

Centralisation

The helder project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the helder project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au